



DOI:10.1145/3631519

Visualizing CI's role in software quality attribute evaluation.

BY LIANG YU, EMIL ALÉGROTH, PANAGIOTA CHATZIPETROU, AND TONY GORSCHKE

A Roadmap for Using Continuous Integration Environments

QUALITY ATTRIBUTES OF software systems,¹³ also known as system qualities, such as performance, security,² and scalability, continue to grow in importance in industrial practice. The evaluation of quality attributes is critical to software development since optimizing a software system's core attributes can provide marketing advantage and set a product apart from its competitors. Many existing studies of unsuccessful development projects report that lack of quality attribute evaluation is often a contributing factor of project failure.^{12,15} Therefore, continuous quality attribute evaluation, throughout the development process, is needed to ensure

customers' expectations and demands⁶ are met.

Manual evaluation of software attributes is common in many software development companies, but it has proven to be insufficient in meeting the demands of rapid releases and high-quality expectations from customers.⁷ Automated practices have therefore gained widespread popularity as a solution to enhance efficiency,¹⁵ reduce costs,⁸ and increase accuracy¹⁵ compared to manual evaluation.

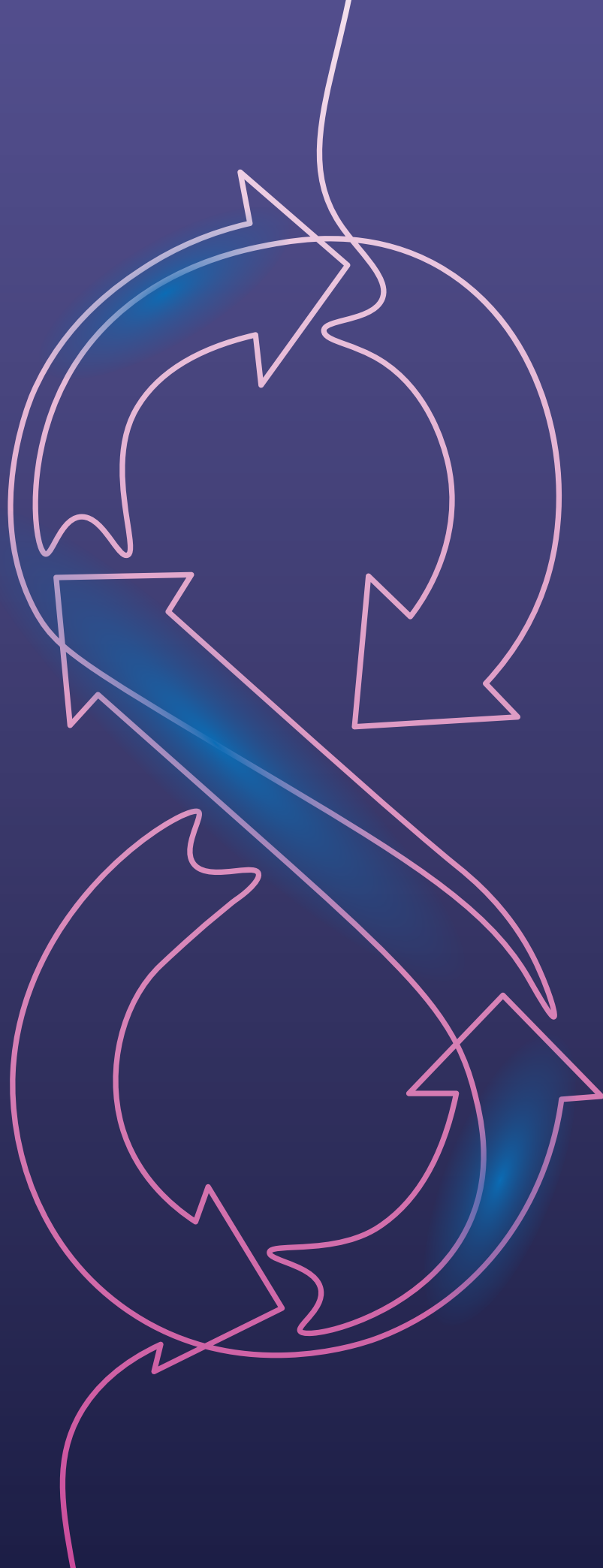
One way to automate the evaluation is using continuous integration (CI)¹⁷ environments. The CI environment provides several benefits, such as fast feedback on code quality, early detection of quality defects, and visualization of system quality trends.¹⁸ As such, these environments inherently offer organizations the opportunity to continuously monitor the quality of their software systems.¹⁹ However, an immature automation process can result in negative outcomes,¹⁹ such as cost and schedule overruns,⁸ slow feedback loops, and delayed releases.⁸

To improve the evaluation process,⁸ prior studies have investigated different key areas, including knowledge,²⁰ processes,⁷ tools,¹² and metrics.³ While leveraging these areas can have a positive impact on quality evaluation, to the best of our knowledge, there is a lack of frameworks that link CI environment knowledge, metrics, and evolution together.

In this article, we aim to fill this gap by presenting the state-of-practice of using CI environments for the evaluation of quality attributes. This is achieved through an industrial study at four partner companies. Study results show that metrics acquired from CI components have a positive effect on evaluating quality requirements. Through analyzing these results, we propose a model by providing guidelines to mature existing CI environments that organizations can use for quality improvements.

As such, we claim the following contributions of this study:

- A generic model of how CI environ-



ments contribute to quality attribute evaluation.

- Empirical evidence that demonstrates how CI components can be used to produce data supporting the evaluation of quality attributes with metrics.

- A model, derived from the study results, which provides decision support to evolve software quality evaluation through CI environments over time.

Study Design

This study aims to gain insights into how CI environments are utilized to evaluate quality attributes in industry, specifically investigating which attributes are commonly evaluated, the contributions of CI environments, and how they can be evolved to suit different evaluation needs.

Case companies. Our case companies are Qvantel Sweden, Ericsson, Fortnox, and Company Alpha.^a Ericsson is a multinational company that produces software and services for telecommunications. The CI environment steers practices for global teams and provides a shared platform with processes and practices for automated verification. Qvantel Sweden is a fast-growing company that provides cloud-native business support services. The CI environment enables faster time to market and flexibility to adapt to changing business needs in terms of quality attribute evaluation. Fortnox offers a cloud-based platform that meets the needs of small businesses and accounting agencies managing their finances efficiently. The CI environment helps developers catch security issues earlier through automated CI jobs. Company Alpha is a Web service provider. The project in this company is a legacy system that concerns businesses using insurance services through Web interfaces. The CI environment increases transparency between software development teams and covers automated performance evaluation.

Research methodology. We designed a qualitative study based on semi-structured interviews. We selected five industrial projects from case companies, and selection criteria includes business domains, number of employees, and types of CI environments, aiming to the best of our abilities to make the study



The continuous integration environment steers practices for global teams and provides a shared platform with processes and practices for automated verification.



as representative of as many software development companies as possible. Business domains cover telecommunications systems, financial services, websites or mobile applications, and customer business support. The projects varied in number of employees, ranging from 50 to 2,500. Five types of CI environments were adopted in these projects to support continuous quality attribute evaluation.

We adopted convenience sampling,²¹ selecting participants from the projects based on their availability and willingness to participate in this study. Our final sample consisted of 22 participants with varying backgrounds in software engineering roles, including managers, software architects, product owners, developers, and testers. However, the distribution across these roles depended on convenience and availability rather than controlled selection. At the time of this study, 86% of selected participants had more than five years of work experience, which could be an indicator of the participants' ability to answer our interview questions.

Data was collected from 22 interviews that were carried out with selected participants. A questionnaire was used as a guide for the interviews, since they had a broad scope, including questions about detailed aspects on CI components, tools, and quality attributes. The evidence was gathered primarily from three of the interview questions, which were:

- How does the existing CI environment look like in your project based on your perception?

- How do you use CI environments for quality attribute evaluation?

- Do you see any challenges in the current processes, techniques, tools, or practices for the evaluation?

Additionally, participants provided related information as part of their responses to other questions, which were also included in the analysis.

Data analysis followed a four-step thematic analysis approach:

Step 1: We coded descriptive information, for example the interview transcripts, using thematic coding.⁴

Step 2: We grouped codes into different categories based on semantic equivalence or a shared contextual relationship.

Step 3: We synthesized codes into

^a Company Alpha is anonymous due to a non-disclosure agreement (NDA).

categories in higher-level themes.⁴

Step 4: We drew conclusions using the synthesized themes. The results were validated through member checking with participants.

How Can CI Environments Contribute to Quality Attribute Evaluation?

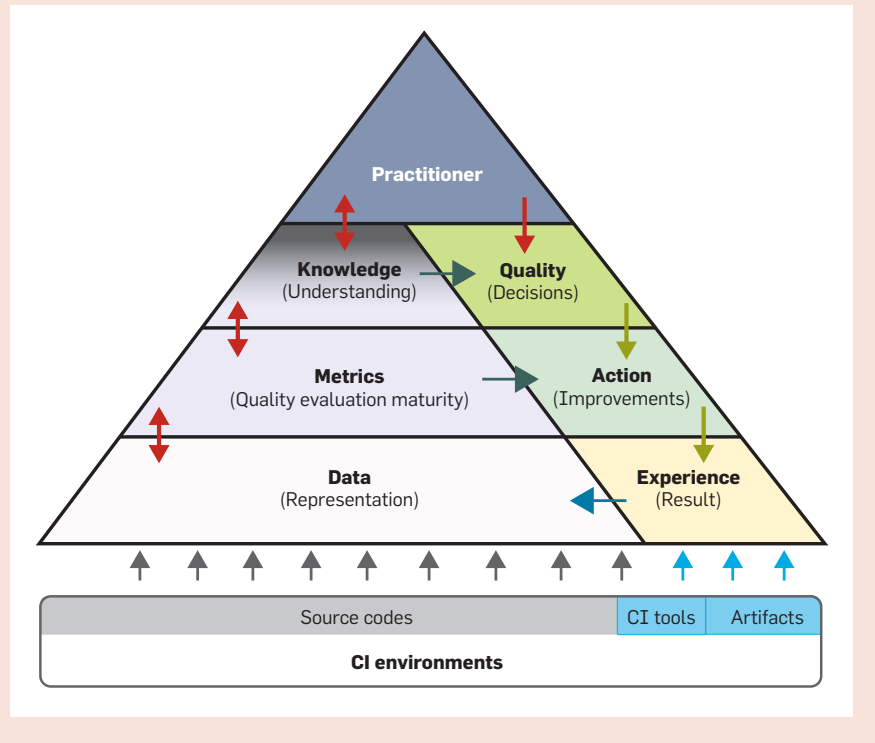
To adequately use any technology or practice, it is essential to have a base framework of understanding.²⁰ Figure 1 presents such a framework, connecting the concept of practitioner knowledge to metrics and data for quality attribute evaluation.

Participants in this study highlighted that CI environments contain various data produced from different sources such as source codes, CI tools, and artifacts (see Figure 1). This data can be extracted through CI jobs and used for continuous evaluation of quality attributes. For instance, an automated vulnerability assessment can be performed by a CI server on source code changes, resulting in measurements of the number of detected vulnerabilities, which in turn supports security evaluation. A deeper understanding/knowledge of the system's attributes supported by the CI process could be helpful for quality managers to make more informed decisions regarding release candidates. This, in turn, enhances the decision-making process from a security perspective. Additionally, leveraging the CI data to inform software release decisions has the potential to optimize the efficiency of the quality attribute evaluation process at the organizational level. Consequently, this can lead to an enhanced experience in collecting data from CI environments. Figure 1 provides a visual representation of this entire process.

Some 73% of the participants in our study reported they added new tools to generate more data, even though much of the data could be extracted from their existing environments, albeit with additional efforts, for example, by using scripts. This highlights the importance for practitioners to be aware of their environments' potential for data generation, otherwise they risk implementing more and more tools over time, as well as new data extraction with overlaps.

Our study also found that knowledge of the environments' capabilities was

Figure 1. Overview of how CI environments contribute to the evaluation of quality attributes.



beneficial in making decisions about improving the environment. For example, to improve the detection of security vulnerabilities, 77% participants mentioned that static code analysis can be adopted as part of their CI process.¹⁸ However, this requires knowledge of how to add such a CI component to the environment and how to effectively use the newly generated data to draw valid conclusions. Therefore, there is a need for general knowledge about CI environments, their components' capabilities, how to instantiate them using tools, what data can be acquired from the tools, and how to use the data effectively.

In summary, 86% of the participants in our study reported that data produced by CI environments could be used to support quality attribute evaluation. To do so effectively, knowledge is required on multiple levels of abstraction including CI tools, data, artifacts, metrics, and experience.

What Is Current State-of-Practice for Using CI Environments for Quality Attribute Evaluation?

We explored the practices of how organizations use CI components for quality attribute evaluation in the selected companies.

Study participants utilized a diverse set of metrics to evaluate Maintainability, Security, Performance, Reliability, Scalability, and Traceability during software development, testing, and release phases. For example, Performance was evaluated by collecting metric data, such as "response time" and "CPU/memory resource usage," from CI components. This form of evaluation through CI-generated data supports a more automated process, which is less error-prone and time-consuming¹⁰ than the manual collection of similar metrics.

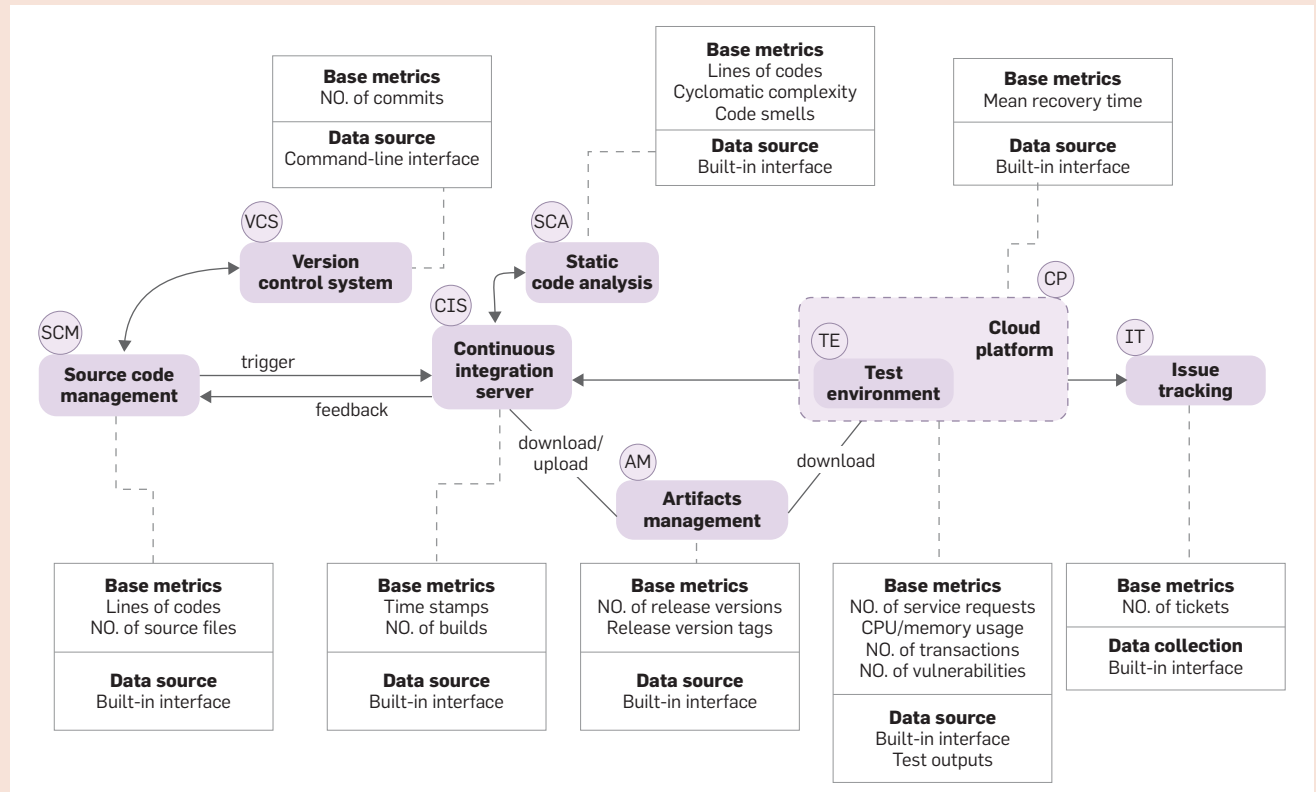
As shown in the accompanying table, we identified seven quality attributes and 10 metrics associated with data and data sources. Moreover, references to relevant literature providing definitions of the identified metrics are also provided. To ensure the validity of our results through data triangulation, we have listed the percentage of study participants who provided information for each metric.

To gain insights into the relationship between metrics and CI components, we focused on gathering base metrics, which are the generic data types generated by the CI components, and their associated data sources. The rationale for focusing on base metrics is that derived

Collected metrics using CI components.

Quality attributes	Metric(s)	Metric data	Data source—CI components	Participants
Maintainability	Percentage of test coverage ¹	Lines of codes	Source code management Static code analysis	50%
		Lines of codes covered by tests	CI server	
	Average cyclomatic complexity on source files ¹¹	Cyclomatic complexity number	Static code analysis	55%
		The number of source files	Source code management	
Security	The total number of vulnerabilities ²	The number of vulnerabilities	CI server	77%
		Vulnerability tickets	Issue tracking	
	Average number of security defects in source codes ¹⁴	Lines of codes	Static code analysis	59%
		The number of defects	Static code analysis	
Performance	Mean Response Time (MRT): ⁹ $MRT = (A_1 + A_2 + \dots + A_n)/n$, where A_i is the response time of a request, and n is the number of requests.	Total response time of a service API request	CI server	82%
		The number of service requests	Test environment	
	Average CPU/memory usage ¹⁸	CPU/memory usage Timestamp	Test environment	73%
Reliability	Rate of failure for building release candidates ³	The number of failed release versions	CI server	18%
		Total number of release versions	Artifacts management	
Scalability	Mean recovery time (MRET): ⁹ $MRET = (A_1 + A_2 + \dots + A_n)/n$, where A_i is the time to recover a service failure, and n is the number of failures.	Total time of fixing failed services	CI server	41%
		The number of service failures	Test environment	
		Mean recovery time	Cloud platform	
Stability	Average transactions per second ⁵	Number of transactions	Test environment	14%
Traceability	Requirement traceability percentage ¹⁶	The number of tickets	Issue tracking	9%
		Total number of tickets	Source code management	

Figure 2. CI components contribute to metric data.



metrics can be calculated by combining the base metrics (see metric definitions in the table). For example, the metric “lines of codes over time” can be referred to as “code churn.”³ As shown in Figure 2, we have identified 14 base metrics.

The data sources used to collect these metrics as reported by the most of participants reflect three main types: built-in interfaces, test outputs, and command-line interfaces. Built-in interfaces, in this case, refer to APIs, or other technical interfaces, in the tools used for a specific CI component, which enable the data to be extracted directly from the tool for further analysis. Test outputs refer to log files or other artifacts that generally require parsing to be further analyzed. Finally, command-line interfaces refer to text-based user interfaces that enable developers to execute automated commands and scripts from a terminal or command prompt for extracting information.

For example, one company in our study utilized nightly CI jobs to evaluate system performance for each release candidate continuously, and the job took approximately four hours. Through this job, developers were able to monitor and compare the system performance (for example, the mean response time of a service request) in code changes (for example, the number of commits) between release candidates. To extract the necessary metrics, they utilized JMeter’s^b built-in interfaces to collect performance results through Rest APIs, and Git^c command-line interfaces, specifically the `git log -since=<time-period> -oneline` command, to get the number of commits during a specific time. When the

performance output in a CI job showed a significant drop, developers were able to either revert their code commits or perform quality analysis to determine the cause of the issue. This approach enabled development teams to identify and address quality issues in a timely and efficient manner.

After understanding what base metrics were available and what data sources can be used to acquire them, we also looked at the use of derived metrics and used a practical example, taken from one of the companies in our study, as a demonstrator, example visualized in Figure 3.

In the example, two types of CI components that produce metric data are introduced. In the first example, the SCM component, instantiated with the tool Gerrit, utilizes a metric exporter plugin (for example, JMX) to collect data (for example, lines of code and source files) and transmits the data using its built-in interface to a data analyzer tool (for example, Prometheus) which is then used to visualize the results in dashboards (for example, in Grafana). In the second example, the test environment (TE) component (for example, tests run in a virtual machine) generates test results in files and logs that are parsed through an in-house developed solution (for example, a parser) to extract data in a format that the data analyzer tool (for example, Prometheus) can read, after which its visualized in the dashboard (for example, in Grafana). These examples highlight that some CI components can take advantage of existing metric exporter tools for data collection, while others may require additional development effort, for example, of specialized data extraction scripts.

The example also provides a practical process for aggregating metric data

into centralized dashboards for quality evaluation. When using centralized dashboards, organizations can gain the following benefits:

- *A single, coherent, overview.* Quality attribute results can be monitored in graphs with trends, providing decision-makers with valuable insights for identifying potential quality-improvement activities or process changes.

- *Flexible monitoring dashboards.* The process supported by CI components enables the creation of various types of monitoring dashboards.

- *Comparative results.* Quality trends can be monitored to see how development/change of one attribute affects the other. For example, improving security through encryption can have a negative impact on system performance.

However, it is important to note that a centralized quality dashboard might bring some challenges that may be considered drawbacks:

- *Quality graphs overflow.* As the number of graphs and dashboards grows, important graphs may be lost in the myriad of graphs and the beneficial overview that was provided is lost. This can result in inaccurate representations of the data and can make it difficult to identify trends or patterns in the data.

- *Insufficient data.* If every metric is visualized, including less accurate ones, it can result in incomplete or inaccurate graphs. This can cause difficulties to draw meaningful conclusions from the data which leads to misinformed decisions.

- *Lack of knowledge about metrics.* If the metric data is misused in the analysis—intentionally or unintentionally—used to derive misleading metrics, it can result in misleading or inaccurate conclusions.

In summary, metrics are beneficial

b <https://jmeter.apache.org/>

c <https://git-scm.com/>

Figure 3. An example of using derived metrics to evaluate quality attributes.

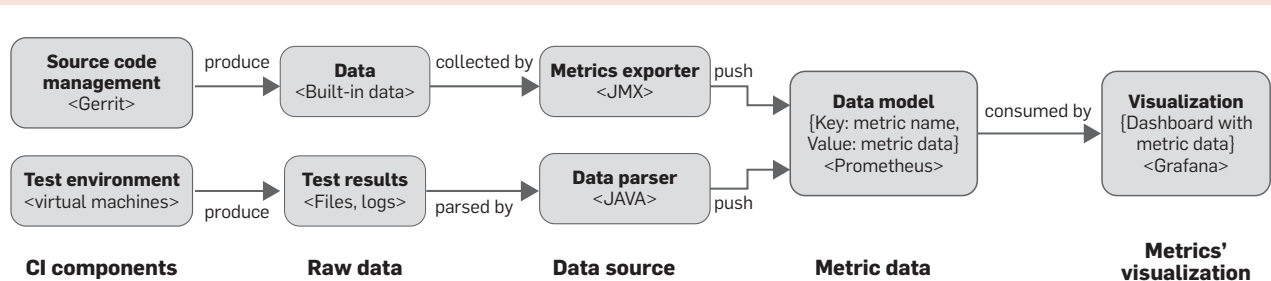
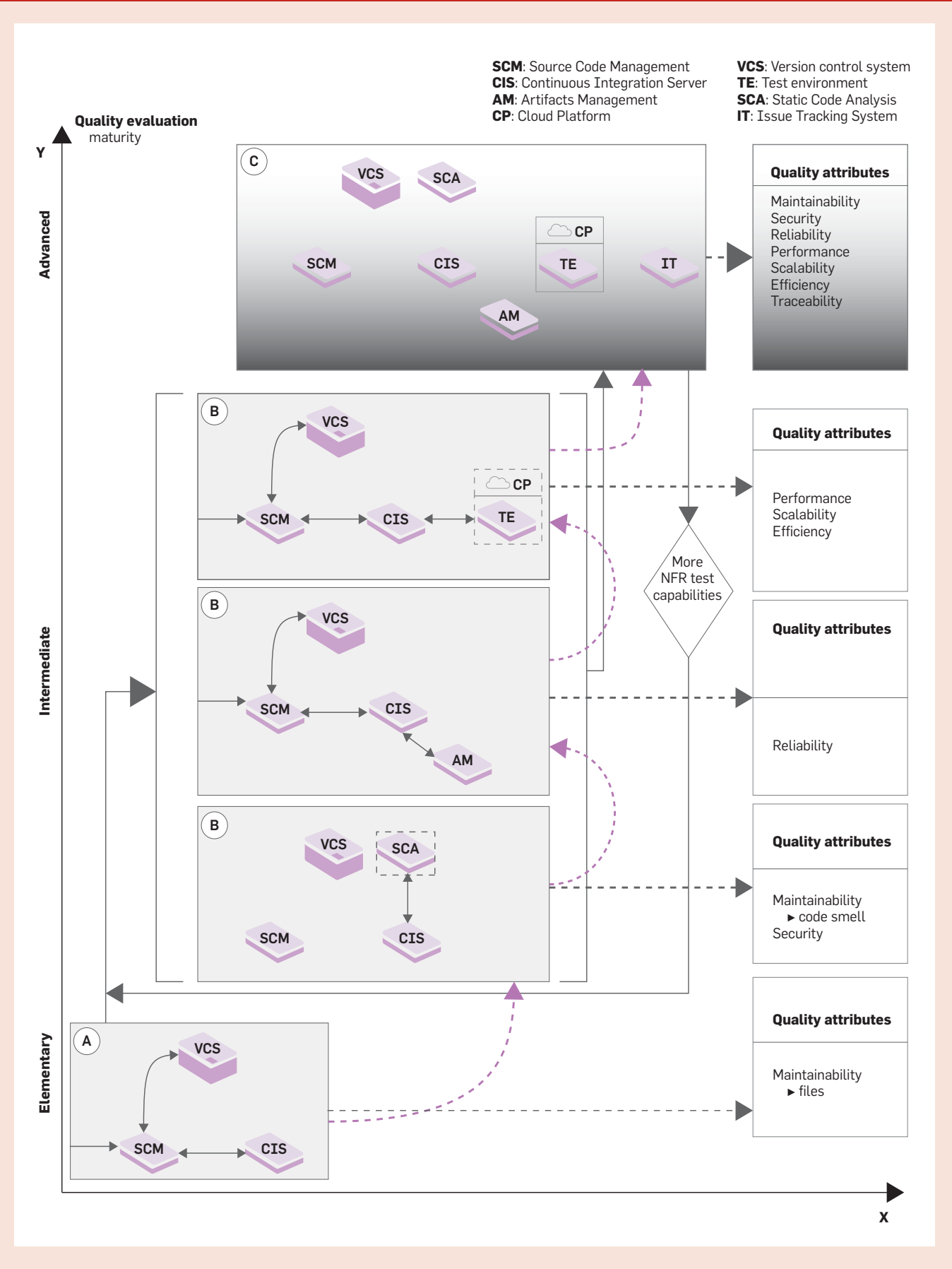


Figure 4. A maturity model for quality attribute evaluation through CI environments.



for organizations to access software quality requirements, potentially providing more objective decision support to drive continuous improvements. To achieve this benefit, it is recommended that practitioners define metrics and acquire a clear understanding of their metric data to mitigate the reported challenges.

How Can Practitioners Evolve CI Environments to Meet the Needs of Quality Attribute Evaluation?

The metrics and CI components used to evaluate quality attributes can vary, depending on the size and maturity of an organization. For example, a small startup may have different needs and resources for quality evaluation compared to a large enterprise organization. Additionally, our study results suggest that there is an evolution from less to more mature CI environments, and this evolution seems to correlate with company growth.

To assist decision-makers in this evolution, we created a maturity model based on our study results. The model was developed through the following steps:

1. *Identification of the relevant metrics for each quality attribute:* We identified a set of metrics relevant for each quality attribute based on the results of our study. For example, for security, we may use metrics like the number of vulnerabilities found and the severity of vulnerabilities.

2. *Mapping of the metrics to the relevant CI components:* When we identified the relevant metrics, we mapped them to the CI components that can produce data to support these metrics. For example, the number of vulnerabilities can be measured using security scanning tools integrated with the CI environment.

3. *Mapping the CI components to the quality attributes:* We first identified the various CI components and their associated metrics. While some metrics produced by a given component may suggest its suitability for evaluating a certain quality attribute, we acknowledge that such a correlation is not always definitive. Therefore, we supplemented our analysis by conducting interviews and making direct observations to gain a deeper understanding of how specific components were used to evaluate specific quality attributes. By



Metrics are beneficial for organizations to access software quality requirements, potentially providing more objective decision support to drive continuous improvement.



triangulating our findings from both the metrics and the qualitative data, we were able to establish a more accurate and reliable mapping between CI components and particular quality attributes for evaluation.

4. *Definition of the maturity levels:* We defined what each maturity level represents after we uncovered the connection between CI components and quality attributes. For example, a basic level of CI maturity for security may mean that security scanning tools are integrated into the CI environment, but vulnerabilities are not being actively addressed. An advanced level of maturity may mean that vulnerabilities are being actively addressed, and security is being continuously improved.

Our proposed model in Figure 4 displays quality evaluation capabilities using CI components in three levels: elementary, intermediate, and advanced. The x-axis represents the increasing needs of assessing quality attributes, while the y-axis represents the evaluation levels.

As shown in the figure, an elementary maturity level can be achieved through employing three CI components to deal with source files' maintainability (see A). When there is a demand for more evaluation capabilities, new CI components (for example, static code analysis) can be added on top of the elementary level to reach an intermediate level (see B). Finally, an advanced maturity level can measure more quality requirements by using additional CI components (see C).

When applying the model, there is no strict order for adding or using several components. This flexibility is highlighted with dash-line boxes (for example, SCA, AM, TE, and CP components) in Figure 4. The selection of CI components can be optimized based on the testing needs of the organization, and our model provides the flexibility to select them to support stepwise improvements in quality evaluation. For instance, if the reliability evaluation of a product is prioritized, the AM component can be added to upgrade the elementary maturity level of an environment to the next level. Additionally, the model also includes a normalized maturity growing path (see dashed and curved connectors in Figure 4), which was identified as common to how the

studied companies have matured their CI environments.

In summary, the level of quality evaluation maturity in a CI environment is associated with both the number of components employed and the extent of quality attribute coverage. Our model offers decision-makers a roadmap to guide the evolving process of their CI environment based on specific quality requirements. It illustrates the trend of how companies, typically, mature in terms of the need to evaluate certain quality attributes and what CI components support such evaluation.

Study Validity

In this study, we acknowledge the relatively small sample size of companies compared to the broader community. As a result, the findings of this study may primarily apply to the specific contexts and company sizes represented in the sample. However, it is crucial to emphasize that the goal of this study is not to achieve extensive generalization. Instead, the focus is on generating valuable evidence that can be integrated with prior research to contribute to a deeper understanding of quality attribute evaluation within the context of CI environments.

Furthermore, given the study is based on evidence, there are several threats that may have impacted the results. For instance, the way in which the interviews were executed or that some participants may not have provided some tacit information. To mitigate this threat, data triangulation process was used.

Conclusion

Our research focused on evaluating the use of CI environments for metrics, specifically looking at how CI components can generate data to support the evaluation of specific quality attributes. Key insights identified through this study are summarized here:

► **Insight 1.** Some 86% of participants reported that CI environments produce data that can aid companies in evaluating specific quality attributes, and the use of metrics can standardize data collection and analysis in CI environments. This highlights the prevalence of utilizing CI environment data for quality attribute evaluation in metrics.

► **Insight 2.** Across the five industrial projects, the number of quality attributes

evaluated through CI environments ranged from 2 to 5, with an average of 3.4 attributes per project. This indicates there is room to expand CI usage for evaluating additional quality attributes.

► **Insight 3.** Almost 73% of study participants stated that centralized dashboards are valuable for visualizing CI metrics data. Reliance on dashboards also underscores their importance for enabling data-driven quality analysis.

► **Insight 4.** On average, study participants had over seven years of experience working with CI environments. This level of expertise is essential for leveraging CI data effectively and highlights the need for knowledgeable practitioners.

► **Insight 5.** This study proposes a maturity model that can aid organizations in assessing their current level of maturity in utilizing CI environments and identifying potential improvements through a roadmap. 60% of selected industrial projects are currently in the intermediate maturity level, which indicates there is room for improvement and standardization of practices in the industry.

In summary, study results revealed trends in how CI environments can be used for quality attribute evaluation across different industrial contexts. Additionally, as more metrics are collected through CI environments, data-driven analysis becomes increasingly more applicable for enhancing the assessment of quality attributes. 

References

- Aggarwal, K.K., Singh, Y., and Chhabra, J.K. An integrated measure of software maintainability. In *Proceedings of the 2002 IEEE Symp. Reliability and Maintainability* (Seattle, WA, USA, 2002), 235–241; 10.1109/RAMS.2002.981648
- Aziz, M.N., Sapta, I.M., and Rochimah, S. Security characteristic evaluation based on ISO/IEC 25023 quality model, case study: Laboratory management information system. In *Proceedings of the 2018 IEEE Electrical Power, Electronics, Communications, Controls, and Informatics Seminar*. (Batu, East Java, Indonesia, 2018), 332–336; 10.1109/EECCIS.2018.8692982
- Colakoglu, F.N., Yazici, A., and Mishra, A. Software product quality metrics: A systematic mapping study. *IEEE Access* 9, (2021), 44647–44670; 10.1109/ACCESS.2021.3054730
- Cruzes, D.S., Dybå, T., Runeson, P., and Höst, M. Case studies synthesis: A thematic, cross-case, and narrative synthesis worked example. *Empirical Softw. Eng.* 20, (2015), 1634–1665.
- del Sagrado, J. and del Águila, I.M. Stability prediction of the software requirements specification. *Softw. Quality J.* 26, 2 (Jun. 2018), 585–605; 10.1007/s11219-017-9362-x
- Fitzgerald, B. and Stol, K.-J. Continuous software engineering: A roadmap and agenda. *J. Systems and Softw.* 123, (Jan. 2017), 176–189; 10.1016/j.jss.2015.06.063
- Garousi, V., Felderer, M., and Hacıoğlu, T. Software test maturity assessment and test process improvement: A multivocal literature review. *Information and Softw. Tech.* 85, (May 2017), 16–42; 10.1016/j.infsof.2017.01.001
- Hilton, M. et al. Usage, costs, and benefits of continuous integration in open-source projects. In *Proceedings of the 31st IEEE/ACM Intern. Conf. Automated Softw. Eng.* (Singapore, 2016), 426–437; 10.1145/2970276.2970358
- ISO, IEC-25023. *Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE): Measurement of System and Software Product Quality*. ISO, Geneva, Switzerland, 2016.
- Kencana, G.H. et al. Comparison of maintainability index measurement from Microsoft CodeLens and line of code. In *Proceedings of the IEEE 2020 7th Intern. Conf. Electrical Eng., Computer Sciences, and Informatics*. (Yogyakarta, Indonesia, 2020), 235–239; 10.23919/EECSI50503.2020.9251901
- Kaynak, I.K., Çilden, E., and Selin Aydın, S. Software quality improvement practices in continuous integration. *Systems, Software and Services Process Improvement*. A. Walker, R.V. O'Connor, and R. Messnarz, Eds. Springer Intern. Publishing, Cham, 2019, 507–517; 10.1007/978-3-030-28005-5_39
- Lin, J.-W., Salehnamadi, N., and Malek, S. Test automation in open-source Android apps: A large-scale empirical study. In *Proceedings of the 35th IEEE/ACM Intern. Conf. Automated Softw. Eng.* ACM, virtual event Australia, 2020, 1078–1089; 10.1145/3324884.3416623
- Mairiza, D., Zowghi, D., and Nurmuliani, N. An investigation into the notion of non-functional requirements. In *Proceedings of the 2010 ACM Symp. Applied Computing*. ACM Press, Sierre, Switzerland, 2010, 311; 10.1145/1774088.1774153
- Morrison, P., Moye, D., Pandita, R., and Williams, L. Mapping the field of software life cycle security metrics. *Information and Softw. Tech.* 102, (Oct. 2018), 146–159; 10.1016/j.infsof.2018.05.011
- Paixao, K.V.R., Felicio, C.Z., Delfim, F.M., and De A. Maia, M. On the interplay between non-functional requirements and builds on continuous integration. In *Proceedings of the 2017 IEEE/ACM 14th Intern. Conf. Mining Softw. Repositories* (Buenos Aires, 2017), 479–482; 10.1109/MSR.2017.33
- Parizi, R.M., Kasem, A., and Abdullah, A. Towards gamification in software traceability: Between test and code artifacts. In *Proceedings of the 10th Intern. Conf. Softw. Eng. and Applications*. Science and Technology Publications, Colmar, Alsace, France, 2015, 393–400; 10.5220/0005555503930400
- Ståhl, D. and Bosch, J. Modeling continuous integration practice differences in industry software development. *J. Systems and Softw.* 87, (Jan. 2014), 48–59; 10.1016/j.jss.2013.08.032
- Wang, Y. et al. Software test automation maturity: A survey of the state of the practice. In *Proceedings of the 15th Intern. Conf. Softw. Technologies, Science and Technology Publications* (Lieuxaint, Paris, France, 2020), 27–38; 10.5220/000976800270038
- Wang, Y. et al. Test automation maturity improves product quality—Quantitative study of open source projects using continuous integration. *J. Systems and Softw.* 188, (Jun. 2022), 111259; 10.1016/j.jss.2022.111259
- Werner, C., Li, Z.S., Ernst, N., and Damian, D. The lack of shared understanding of non-functional requirements in continuous software engineering: Accidental or essential? In *Proceedings of the IEEE 28th Intern. Requirements Engineering Conf.* (Zurich, Switzerland, 2020), 90–101; 10.1109/RE48521.2020.00021
- Wohlin, C. *Experimentation in Software Engineering*. Springer, New York, 2012.

Liang Yu is Ph.D. student at Blekinge Institute of Technology, Sweden.

Emil Alégroth is a professor in the Department of Software Engineering at Blekinge Institute of Technology, Sweden.

Panagiota Chatzipetrou is an associate professor in the Department of Informatics at Örebro University, Sweden.

Tony Gorschek is a professor of software engineering at Blekinge Institute of Technology, Sweden and Fortiss GmbH, Germany.

© 2024 Copyright held by the owner/author(s).